

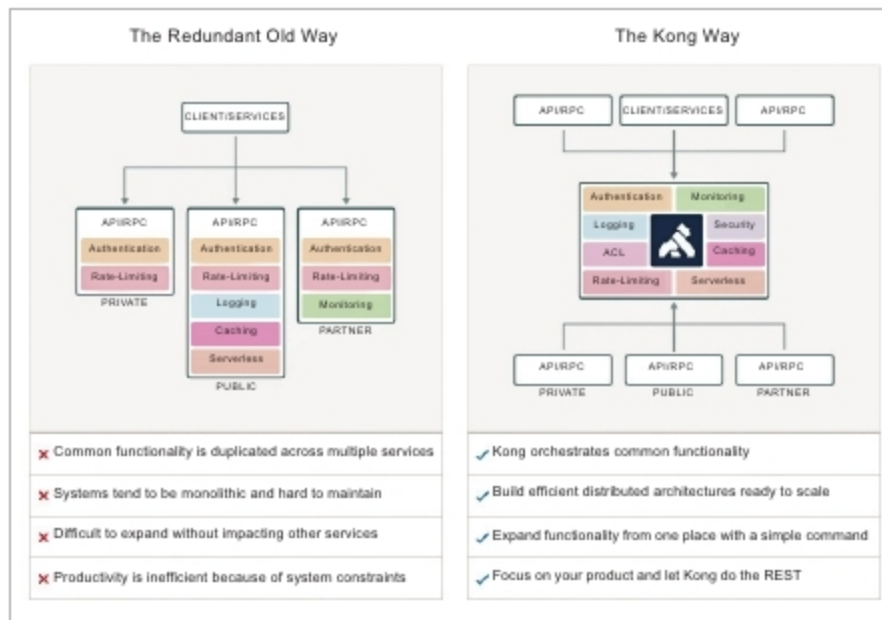


Figure 4: Kong API gateway

The Kong API gateway

The Kong API gateway acts as the middleware between computing clients and API-based applications. There are many advantages of using the Kong API gateway. It is extremely fast, RESTful and platform agnostic. Kong has many features and it can be extended by adding plugins for authentication, security, traffic control, logging, monitoring and analytics purposes. Kong is available as a community edition as well as an enterprise edition to address the need to build microservice API gateway platforms. Kong can be set up and scaled on any platform -- on-premise, the cloud or on a hybrid one. It is very easy to set up in a local machine. If you wish to evaluate the product before using it, please spin up a VM using AMI from the AWS marketplace.

A typical Kong setup comprises two main components:

1. Kong's server based on the widely adopted Nginx HTTP server, which is a reverse proxy that processes clients' requests to your upstream services.
2. Kong's data store, in which the configuration is stored to allow the horizontal scaling of Kong nodes. Apache Cassandra and PostgreSQL can be used to fulfil this role.

Kong needs to have both these components set up and operational.

The Kong community version supports CLI and the admin API for managing Kong instances. But it does not have an admin GUI for management. Third party tools such as the Kong dashboard support the GUI for managing Kong instances. The Kong enterprise version has full support including the admin dashboard.

The Kong admin API can be accessed by Port 8001. Let us create a service and add routes to access the service through the Kong API gateway. Here, the service is `http://microservices.io/scripts/bootstrap.js`. Instead of accessing the service directly, it should be registered to the Kong API gateway. It will create an upstream server in Nginx, the configuration for which is maintained in a data store.

A service is the name Kong uses to refer to the upstream APIs and microservices it manages. Before one can start

making requests against the service, you need to add a route to it. Routes specify how (and if) requests are sent to their services after they reach Kong. A single service can have many routes.

#Create a service in Kong API Gateway

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"name": "microservice0", "host": "microservices.io", "port": 80, "protocol": "http", "path": "/scripts/bootstrap.js"}' \
  http://18.212.88.185:8001/services/
```

#Add a route to access the service

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"hosts": ["microservices.io"]}' \
  --url http://18.212.88.185:8001/services/microservice0/routes
```

#Access the resource and ensure Kong is properly forwarding request to the service.

```
curl --header "Content-Type: application/json" \
  --header "Host: microservices.io" \
  --request GET \
  --url http://18.212.88.185:8000/
```

#List all registered services

```
curl --header "Content-Type: application/json" \
  --request GET \
  --url http://18.212.88.185:8001/services
```

#List all routes

```
curl --header "Content-Type: application/json" \
  --request GET \
  --url http://18.212.88.185:8001/routes
```

#Enable authentication to the service

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"name": "key-auth"}' \
  --url http://18.212.88.185:8001/services/microservice0/plugins
```

#Ensure Kong API Gateway will throw an error for unauthorized access of resources

```
curl --header "Content-Type: application/json" \
  --header "Host: microservices.io" \
  --request GET \
  --url http://18.212.88.185:8000/
```

#Create a consumer to access the service with authentication

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"username": "rkkrishnaa"}' \
```